32-Data Visualization with Matplotlib and Seaborn

Introduction

- The session covers Python's Matplotlib and Seaborn libraries for data visualization, with practical demonstrations and stepwise explanation.

Matplotlib Overview

- Matplotlib is primarily used for 2D plotting, with limited 3D capabilities.
- Many visualization libraries (e.g., Seaborn, Plotly) are built on top of Matplotlib.
- Important for controlling plot aesthetics (color, size, style).
- Modules discussed: `matplotlib.pyplot` (typically imported as `plt`).

Basic Plotting Approaches

- Two main approaches: functional (pyplot) and object-oriented (OO).
  - Functional approach: Directly use `plt.plot()` with arrays.
  - OO approach: Create Figure and Axes objects for more control and customization.
- Explained x/y labeling, titles, and showing/saving plots via `plt.show()` and `fig.savefig()`.

Customizing Plots

- Can adjust plot size using `figsize` and resolution using DPI.
- Figures and axes: A figure can host multiple axes (plots).
- Subplots: Using `plt.subplot()` (for functional) and `plt.subplots()` (for OO) to create multi-plot grids.
- Detailed indexing for accessing subplots in a grid.

Multiple Plots and Subplots

- `plt.subplot()` for placing multiple plots on a single canvas—specified by (rows, columns, index).
- Demonstrated both functional and OO approaches for multiple plots/managing axes.
- `plt.subplots()` returns both Figure and Axes objects, automating axes management.

Saving and Exporting Figures

- Saving plots with `fig.savefig('filename.png')`.
- DPI (dots per inch) parameter to control output quality.

Legends and Colors

- Legends differentiate multiple data series within one Axes; location managed by `ax.legend(loc=...)`.
- Demonstrates automatic placement, and manual values for legend location.
- Lists color code abbreviations and support for hexadecimal color codes.

Line and Marker Customization

- Customizing line width (`linewidth`/`lw`), line style (`linestyle`/`ls`), and marker styles (`marker`).
- Shows usage of shape, size, edge/face color, edge width for markers.

Introduction to Seaborn

- Seaborn is built atop Matplotlib, importing it requires Matplotlib to be present.
- Requires a data set (e.g., example with `pokemon.csv`).
- Seaborn simplifies plot creation, especially for statistical and distribution visualizations.

Distribution and Density Plots

- Univariate analysis with `sns.distplot`, providing histogram and KDE options.
- Setting axis labels and titles, managing histogram/kde toggling.

Rug and Histogram Plots

- `sns.rugplot` displays value distribution along an axis.
- Binning and exact value counts explored with histogram bin size.
- Demonstrates using value counts to get exact repetition of data points.

Bivariate/Joint Plots

- Joint plots (`sns.jointplot`) visualize distributions of two variables (scatter, hex, KDE, regression plots).
- Scatter plots show distribution; can switch kind of plot (scatter, hexbin, KDE, regression).

Heatmaps and Correlation

- Use of pandas' correlation matrix and Seaborn's heatmap (`sns.heatmap`) to visualize correlations between numerical columns.
- Customizations: colormap (cmap), annotation formatting, max-value (vmax), annot toggling.

Regression and Pair Plots

- Regression plots (`sns.lmplot`) show linear relationships between variables, with regression line and confidence intervals.
- Supports dimension splitting using hue and col attributes.

- Pair plots (`sns.pairplot`) for multi-variable exploratory analysis, visualizing relationships for many variable pairs simultaneously. Kind parameter toggles between scatter, regression, etc.

Exam/Assignment Notes

- Answers to practical assignments available on Google Drive.
- Upcoming exam will cover mostly NumPy and Pandas, not Matplotlib/Seaborn.
- Exam will include both MCQs and programming questions.
- Topics from Scikit-learn and advanced libraries will be discussed later in the course.

Additional Notes

- Students asked various clarifying questions about syntax and functionality.
- Instructor explained the logic behind internal attributes, and how to control and interpret visual output.
- Emphasized learning structure and methods, with practical case studies to follow.